

SE3082 – Parallel Computing
Assignment 03
BSc (Hons) in Computer Science
Year 3 Semester 1 – 2026

Name: Mohamed Aslam Amaan Shafie
Student ID: IT23657250

1. Introduction

Parallel computing is used a lot to boost the performance of computationally heavy applications, by doing several operations at the same time. Nowadays applications in graph analytics, scientific modeling, and machine learning, usually need huge amounts of data processing which often goes beyond what a pure sequential approach can handle.

In this project, we look at the parallelization of the PageRank algorithm, it's a graph based ranking method that was originally crafted for ordering web pages. PageRank works in an iterative manner and it estimates how important each node is in a graph, based on the connections and link structure between nodes. Because the rank updates for different nodes can be handled independently, the method feels pretty natural for parallel execution.

The goal of this assignment is to implement and then assess both serial and parallel variants of the PageRank algorithm, using OpenMP, MPI, and CUDA technologies. We analyze performance markers like run time and speedup, so we can compare how effective each parallel programming model really is.

2. Algorithm Background

The PageRank method sort of imagines a random web wanderer going through linked pages, you know, like a little click trail that isn't entirely predictable. For every node, the value it owns gets shared out evenly across its outgoing links and then the system keeps recalculating those node scores over and over, until it basically settles down into a stable arrangement. There's also a damping factor in there, which mirrors the chance that a user will suddenly hop somewhere else, instead of just following the hyperlinks in a straight line.

3. Serial Implementation

The serial implementation figures out PageRank values in a sequential way, sort of with nested loops all over. It uses an adjacency matrix to stand for graph connections. In every iteration, each node gathers up contributions from its in bound neighbors first, then it updates its own rank value. And yeah, the accumulation part happens before the new rank, almost like, step by step without any parallelism, really.

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03$ cd Serial/
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/Serial$ gcc pagerank_serial.c -o pagerank_serial
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/Serial$ ./pagerank_serial

===== SERIAL PAGERANK =====
Nodes       : 1000
Iterations  : 100
Execution Time : 0.368299 seconds

Sample PageRank Values:
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/Serial$
```

4. OpenMP Parallelization

The OpenMP implementation parallelizes the outer loop that is in charge of updating the node ranks, like that. Because each node's rank calculation is independent during a given iteration, a bunch of threads can safely churn through PageRank computations at the same time, so there's no real interference. The OpenMP directives also got used to split up the loop iterations across the available CPU threads. With shared memory, the workers could access the data efficiently with very low communication overhead, not much talking between them, basically.

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ export OMP_NUM_THREADS=1
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ ./pagerank_openmp

===== OPENMP PAGERANK =====
Nodes       : 1000
Iterations  : 100
Threads Used : 1
Execution Time : 0.354943 seconds

Sample PageRank Values:
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ export OMP_NUM_THREADS=2
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ ./pagerank_openmp
```

```
===== OPENMP PAGERANK =====
Nodes       : 1000
Iterations   : 100
Threads Used : 2
Execution Time : 0.237905 seconds
```

Sample PageRank Values:

```
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ export OMP_NUM_THREADS=4
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ ./pagerank_openmp
```

```
===== OPENMP PAGERANK =====
Nodes       : 1000
Iterations   : 100
Threads Used : 4
Execution Time : 0.209672 seconds
```

Sample PageRank Values:

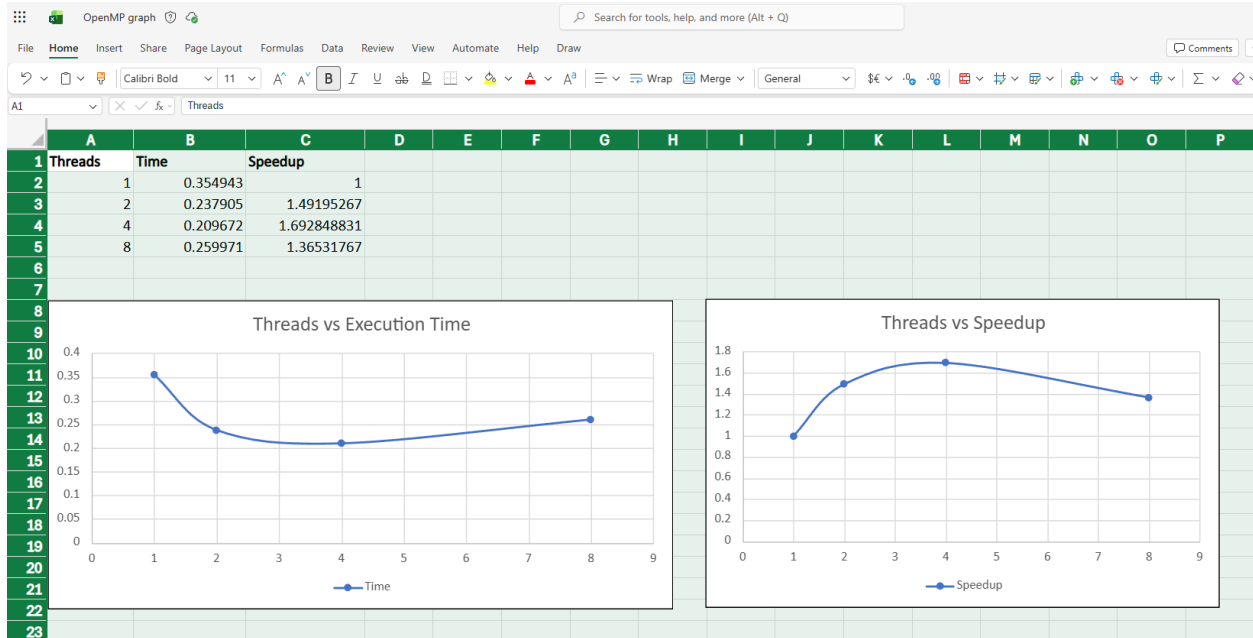
```
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ export OMP_NUM_THREADS=8
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/OpenMP$ ./pagerank_openmp
```

```
===== OPENMP PAGERANK =====
Nodes       : 1000
Iterations   : 100
Threads Used : 8
Execution Time : 0.259971 seconds
```

Sample PageRank Values:

```
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```



5. MPI Parallelization

In the MPI implementation, graph nodes were kind of split equally among the processes, maybe more or less. Each process then ran PageRank calculations for the nodes subset it got, locally. After those computations, MPI_Allgather kicked in, so updated rank values could be shared and synced across the whole job, all processes at once. With this scheme, you still get distributed-memory parallelism while also keeping the results correct.

```

amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03$ cd MPI/
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/MPI$ ls
pagerank_mpi.c
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/MPI$ mpicc pagerank_mpi.c -o pagerank_mpi
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/MPI$ mpirun -np 1 ./pagerank_mpi

===== MPI PAGERANK =====
Processes Used : 1
Nodes : 1000
Iterations : 100
Execution Time : 0.325448 seconds

Sample PageRank Values:
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986

```

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/MPI$ mpirun -np 2 ./pagerank_mpi

===== MPI PAGERANK =====
Processes Used : 2
Nodes          : 1000
Iterations     : 100
Execution Time : 0.204134 seconds

Sample PageRank Values:
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/MPI$ mpirun -np 4 ./pagerank_mpi

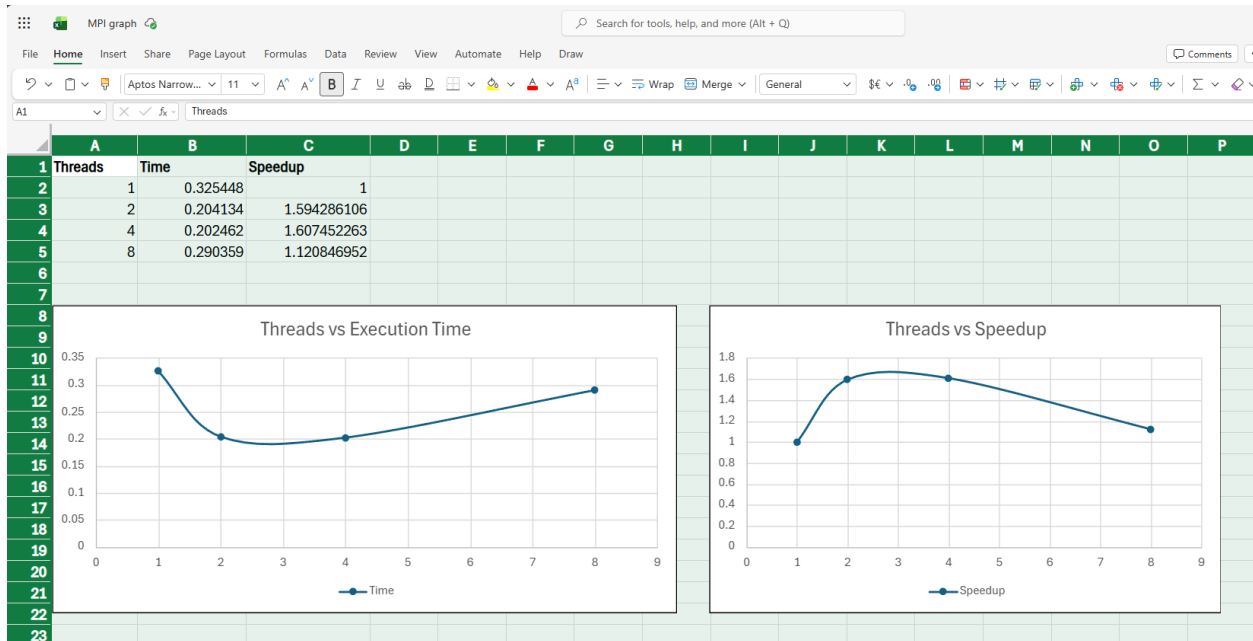
===== MPI PAGERANK =====
Processes Used : 4
Nodes          : 1000
Iterations     : 100
Execution Time : 0.202462 seconds

Sample PageRank Values:
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```

```
amaan@LAPTOP-EC6DJSIA:/mnt/c/Users/nisath/Dropbox/PC/Desktop/PageRank-Algo/SE3082_Assignment03/MPI$ mpirun -np 8 ./pagerank_mpi

===== MPI PAGERANK =====
Processes Used : 8
Nodes          : 1000
Iterations     : 100
Execution Time : 0.290359 seconds

Sample PageRank Values:
Node 0 : 0.001198
Node 1 : 0.000864
Node 2 : 0.001281
Node 3 : 0.001117
Node 4 : 0.001143
Node 5 : 0.001031
Node 6 : 0.001324
Node 7 : 0.000955
Node 8 : 0.001021
Node 9 : 0.000986
```



6. CUDA Parallelization

The CUDA setup used GPU parallelism to speed up PageRank pretty a lot. A CUDA kernel was built, and each GPU thread just computed the rank of one node. So thousands of those threads ran at the same time, sort of concurrently, and it cut the execution time a ton compared with the CPU based implementations. Also, we tried various thread block configurations, to see what was the best fit for performance on the GPU.

```
[1] !nvidia-smi
✓ Os
... Sat May 9 08:05:07 2026
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0      |
+-----+-----+-----+-----+-----+-----+
| GPU  Name          Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+-----+
|  0  Tesla T4             Off      | 00000000:00:04.0 Off  |    0%      Default  |
| N/A   54C    P8             10W / 70W |  0MiB / 15360MiB |           |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                  GPU Memory |
|   ID   ID                               Name                          Usage     |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+
```

Overwriting pagerank_cuda.cu

!nvcc pagerank_cuda.cu -o pagerank_cuda

... nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will

!./pagerank_cuda

...

===== CUDA PAGERANK =====

Threads Per Block : 256

Execution Time : 249.143524 ms

Sample Values:

Node 0 : 0.001198

Node 1 : 0.000864

Node 2 : 0.001281

Node 3 : 0.001117

Node 4 : 0.001143

Node 5 : 0.001031

Node 6 : 0.001324

Node 7 : 0.000955

Node 8 : 0.001021

Node 9 : 0.000986

!./pagerank_cuda

...

===== CUDA PAGERANK =====

Threads Per Block : 32

Execution Time : 108.376320 ms

Sample Values:

Node 0 : 0.001198

Node 1 : 0.000864

Node 2 : 0.001281

Node 3 : 0.001117

Node 4 : 0.001143

Node 5 : 0.001031

Node 6 : 0.001324

Node 7 : 0.000955

Node 8 : 0.001021

Node 9 : 0.000986



!./pagerank_cuda

...

===== CUDA PAGERANK =====

Threads Per Block : 64

Execution Time : 74.321922 ms

Sample Values:

Node 0 : 0.001198

Node 1 : 0.000864

Node 2 : 0.001281

Node 3 : 0.001117

Node 4 : 0.001143

Node 5 : 0.001031

Node 6 : 0.001324

Node 7 : 0.000955

Node 8 : 0.001021

Node 9 : 0.000986



!./pagerank_cuda

...

===== CUDA PAGERANK =====

Threads Per Block : 128

Execution Time : 105.865059 ms

Sample Values:

Node 0 : 0.001198

Node 1 : 0.000864

Node 2 : 0.001281

Node 3 : 0.001117

Node 4 : 0.001143

Node 5 : 0.001031

Node 6 : 0.001324

Node 7 : 0.000955

Node 8 : 0.001021

Node 9 : 0.000986



!./pagerank_cuda

...

===== CUDA PAGERANK =====

Threads Per Block : 256

Execution Time : 109.306366 ms

Sample Values:

Node 0 : 0.001198

Node 1 : 0.000864

Node 2 : 0.001281

Node 3 : 0.001117

Node 4 : 0.001143

Node 5 : 0.001031

Node 6 : 0.001324

Node 7 : 0.000955

Node 8 : 0.001021

Node 9 : 0.000986



!./pagerank_cuda

...

===== CUDA PAGERANK =====

Threads Per Block : 512

Execution Time : 196.026245 ms

Sample Values:

Node 0 : 0.001198

Node 1 : 0.000864

Node 2 : 0.001281

Node 3 : 0.001117

Node 4 : 0.001143

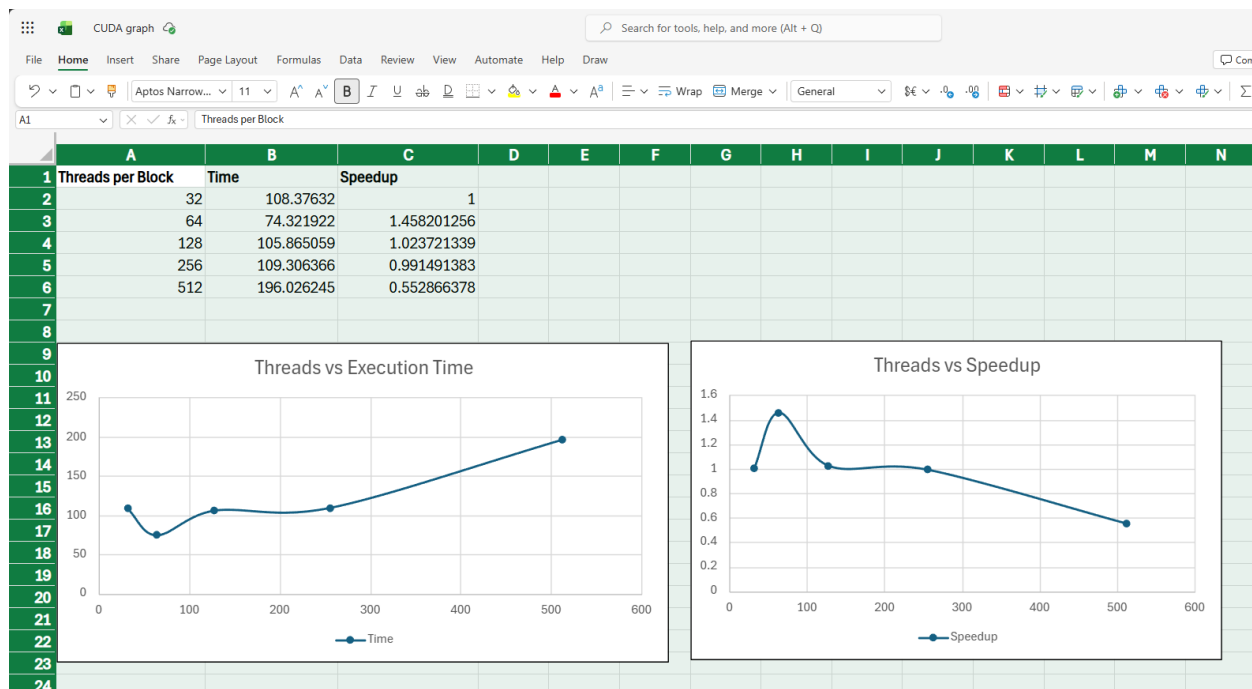
Node 5 : 0.001031

Node 6 : 0.001324

Node 7 : 0.000955

Node 8 : 0.001021

Node 9 : 0.000986

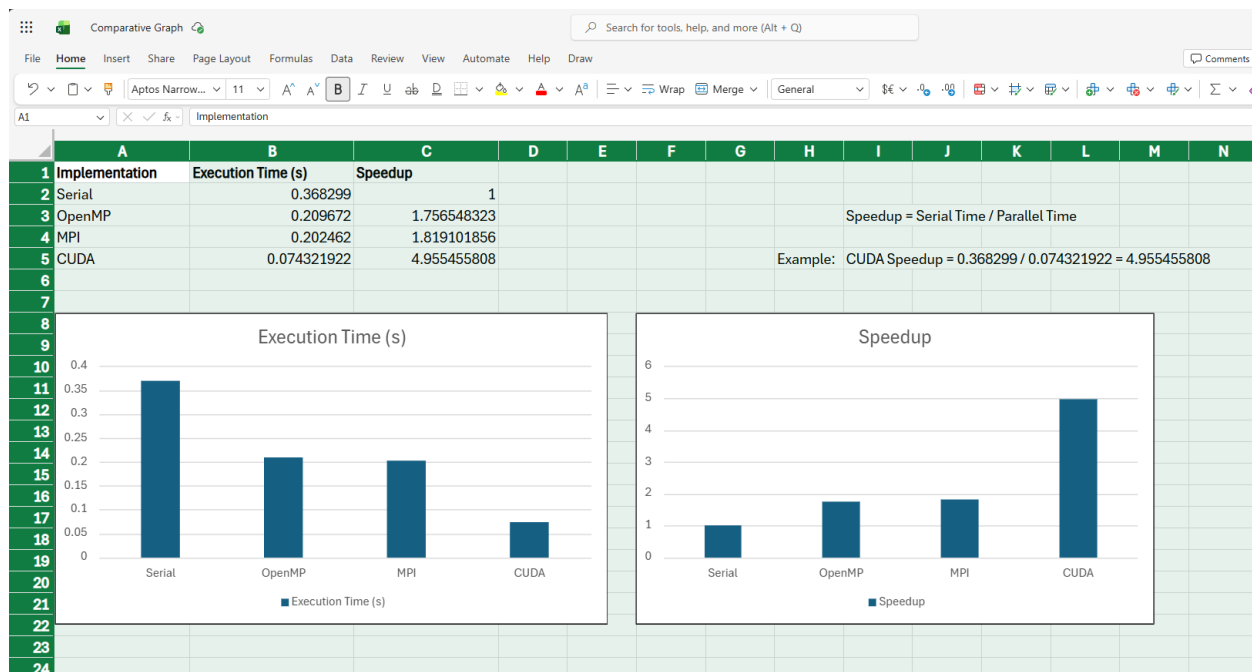


7. Performance Evaluation

Experimental outcomes showed pretty big gains in performance for every parallel setup , compared with the serial baseline. In OpenMP, shared-memory parallelism ran quite efficiently, and the speedup was decent but not earth-shattering. With MPI, distributed execution scaled well across nodes yet, at larger process counts it brought in extra communication time , which kind of spoiled things a bit. For CUDA, the results ended up being the best, since massive GPU parallelism really did the heavy lifting.

8. Comparative Analysis

In all the implementations, CUDA sort of gave the best overall performance, since the GPU could run thousands of PageRank computations in parallel at once. OpenMP still looked good for CPU parallelism, with low synchronization overhead and a fairly smooth execution. MPI, on the other hand, made it possible to spread the work across multiple processes and machines but it also brought in extra communication costs, which kind of slows things down. The overall results suggest that GPU acceleration is the more suitable option for large scale PageRank runs, provided there are enough GPU resources available.



9. Challenges and Limitations

During the actual implementation, a few challenges came up unexpectedly. The MPI communications overhead did end up affecting the scaling quite a lot, especially when the number of processes got higher. Also, CUDA work wasn't just plug in and go—it took careful memory management and a lot of tuning, mostly around thread configuration, timing and so on. On top of that, load imbalance, and those memory bandwidth limitations, really hurt performance in some of the configurations we tried.

10. Conclusion

This project kind of managed to implement both serial and parallel PageRank versions, and did it with OpenMP, MPI, and CUDA as well. So, performance tests showed that running things in parallel boosts the computational efficiency quite a lot, especially for graph analytics tasks. For the CUDA part, it actually got the highest speedup, which kind of points at how well GPU oriented parallelism works for these big iterative algorithms, you know, the ones that repeat over and over.

11. References

- [1] L. Page, S. Brin, R. Motwani, and T. Winograd,
"The PageRank Citation Ranking: Bringing Order to the Web,"
Stanford InfoLab, 1999.

- [2] B. Chapman, G. Jost, and R. van der Pas,
Using OpenMP: Portable Shared Memory Parallel Programming,
MIT Press, 2008.

- [3] W. Gropp, E. Lusk, and A. Skjellum,
Using MPI: Portable Parallel Programming with the Message Passing Interface,
MIT Press, 2014.

- [4] NVIDIA Corporation,
CUDA C Programming Guide, 2024.